COSC 3500 Milestone 1

By Aryan Dhotre
s4985050

Forest Fire Cellular Automata

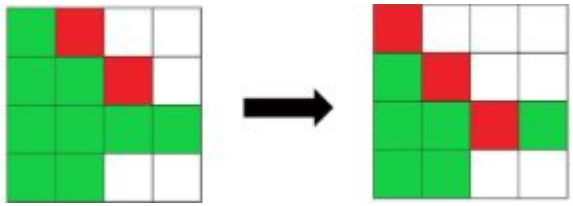
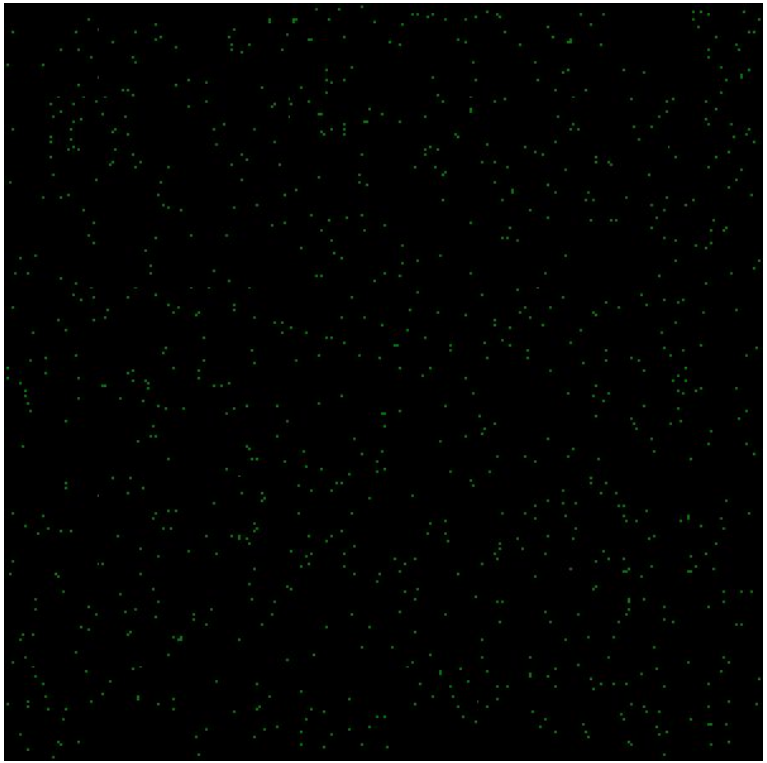


Fig.1. One timestep of forest fire model. Source: [1].

1. A tree grows on an empty cell with a probability p
2. A tree ignites with a probability f
3. A tree will burn if one of its neighbors is on fire
4. A burning cell turn into an empty cell



[1] “Forest-fire Model.” Accessed: Aug. 27, 2026. [Online]. Available: https://itp.uni-frankfurt.de/~gros/StudentProjects/Projects_2020/projekt_lars_dingeldein/
[2] B. Drossel and F. Schwabl, “Self-organized critical forest-fire model,” Phys. Rev. Lett., vol. 69, no. 11, pp. 1629–1632, Sep. 1992, doi: 10.1103/PhysRevLett.69.1629.

Benchmarking methodology

- git
- `time`
- tests at o1 and o2
- tests for 100x100 , 500 x 500 , 1000 x 1000 grid size
- each test ran 5 times
- both o1 and o2 are tested within the same job
- slurm script prints the hardware configuration at the start of the job

data/v0.2/slurm-logs/slurm-o1-o2.out

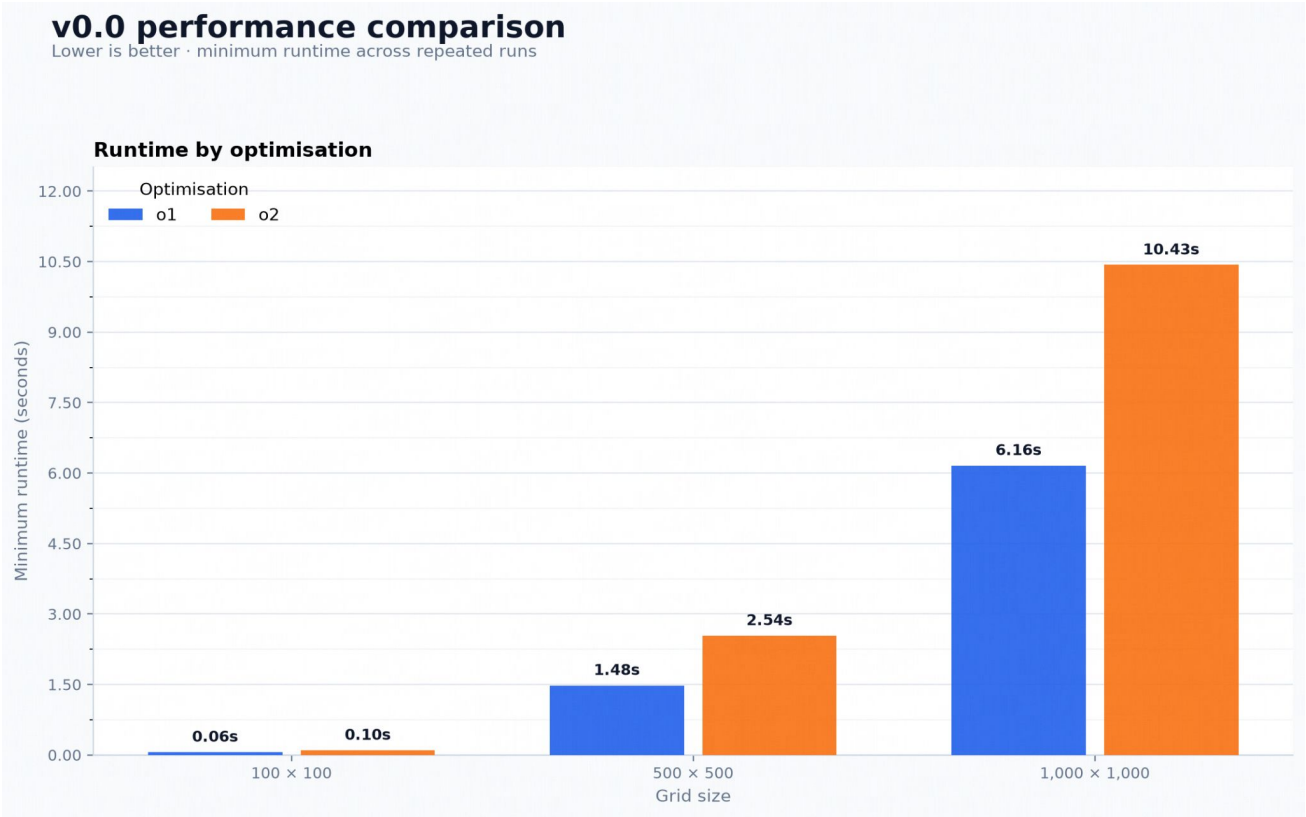
```
1   == execution environment ==
2   opt flags: o1 o2
3   job id:   563969
4   job node: vcpu-2
5   hostname: vcpu-2
6   kernel:   4.18.0-553.150.1.el8_10.x86_64
7   date:     2026-08-23T18:03:07+10:00
8   working directory: /home/Student/s4985050/forest-fire-sim
9   command: /var/slurm/d/job563969/slurm_script o1,o2 v0.2
10  parameters: grids=100 500 1000 timesteps=500 warmups=2 reps=5 p=0.02 f=0.01
11  slurm allocation: job_cpus_per_node=1 cpus_per_task=1 ntasks=1
12  process affinity: 0
13  load average: 0.02 0.33 0.42 1/308 366773
14  uptime:  18:03:07 up 20 days,  8:04,  0 users,  load average: 0.02, 0.33, 0.42
15  Architecture:        x86_64
16  CPU(s):              8
17  Thread(s) per core:  1
18  Core(s) per socket:  1
19  Socket(s):           8
20  NUMA node(s):        1
21  Model name:          AMD EPYC 7542 32-Core Processor
22  Virtualization type: full
23  NUMA node0 CPU(s):   0-7
24  model name  : AMD EPYC 7542 32-Core Processor
25  pinned core: 0
26  cpufreq governor: n/a
27  cpufreq current kHz: n/a
28  compiler command: /usr/bin/g++
29  g++ (GCC) 8.5.0 20210514 (Red Hat 8.5.0-28)
30  time command: time (GNU Time) UNKNOWN
```

Program startup

Each binary prints which model configuration it's ran with , the version number and version name. It also identifies the gcc and c++ version

```
Cellular automata based forest fire model
-------------------------------------------------
Model configuration
  Grid size:       1002
  Time steps:      500
  p (growth):      0.02
  f (lightning):   0.01
-------------------------------------------------
Build information
  Version          0.2 better-random
  Compiled:        Aug 23 2026 18:02:53
  Compiler:        GCC 8.5.0 20210514 (Red Hat 8.5.0-28)
  C++ standard:    C++17 (__cplusplus=201703)
  Compiler target: x86_64-redhat-linux
  Executable:      bin/fire-v0.2-o2
  Build flags:     O2 NDEBUG
  Optimisation:    enabled
```

v0.0


v0.0 performance comparison
Lower is better · minimum runtime across repeated runs
Runtime by optimisation
Optimisation
o1
o2
Minimum runtime (seconds)
12.00
10.50
9.00
7.50
6.00
4.50
3.00
1.50
0.00
0.06s
0.10s
1.48s
2.54s
6.16s
10.43s
100 × 100
500 × 500
1,000 × 1,000
Grid size

v0.0 - running gprofng

data/v0.0/gprofng/o2.txt

```
Functions sorted by metric: Exclusive Total CPU Time

Excl. Total     Incl. Total     Name
CPU             CPU
  sec.      %     sec.      %
10.287 100.00  10.287 100.00   <Total>
10.287 100.00  10.287 100.00   main
 0.      0.    10.287 100.00   __libc_start_main
```

main.cpp

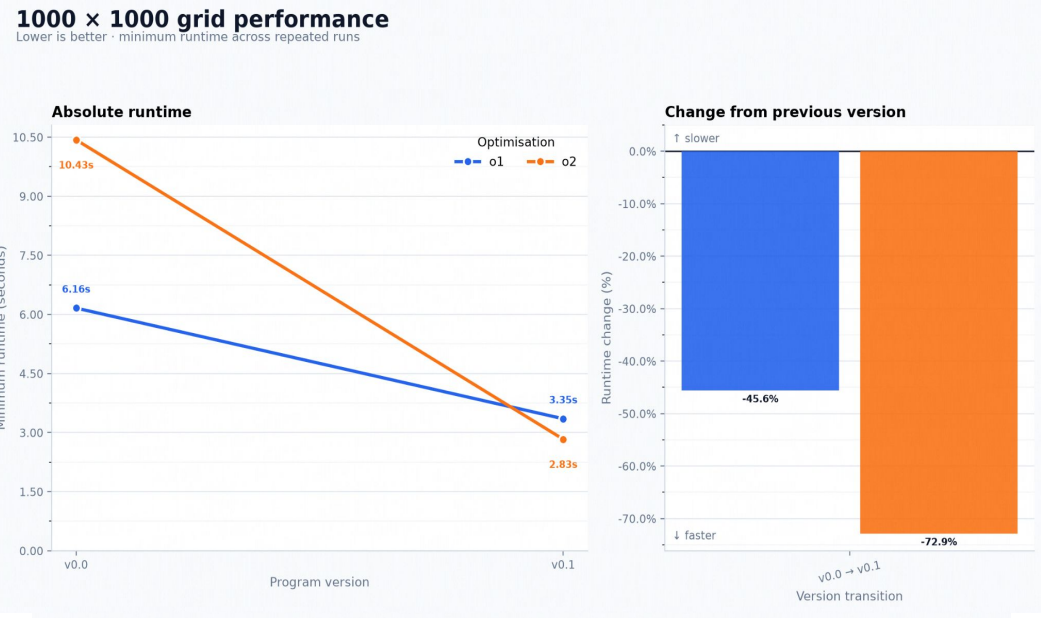
```
74    std::mt19937 gen(80085);
75    std::uniform_real_distribution<double> dist(0.0, 1.0);
76
77    // TODO : compute the hash of this data structure
78    std::vector<std::vector<CellState>> grid(grid_size,
79                                             std::vector<CellState>(grid_size));
80
81    std::vector<std::vector<CellState>> next(grid_size,
82                                             std::vector<CellState>(grid_size));
83    // TODO : change this to fix the cache access patterns
84    for (int t = 0; t < timesteps; ++t) {
85      for (int y = 1; y < grid_size - 1; ++y) {
86        for (int x = 1; x < grid_size - 1; ++x) {
87          auto r = dist(gen);
88          switch (grid[x][y]) {
89          case CellState::Empty:
90            if (r < p) {
91              next[x][y] = CellState::Tree;
92            } else {
93              next[x][y] = CellState::Empty;
94            }
95            break;
96          case CellState::Burning:
97            next[x][y] = CellState::Empty;
98            break;
99          case CellState::Tree:
100           const auto up = grid[x][y - 1];
```

```
Functions sorted by metric: Exclusive Total CPU Time

Excl. Total     Incl. Total     Name
CPU             CPU
 sec.      %    sec.      %
6.114 100.00   6.114 100.00   <Total>
3.823  62.52   6.114 100.00   main
2.292  37.48   2.292  37.48   std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 11ul, 4294967295ul,
7ul, 2636928640ul, 15ul, 4022730752ul, 18ul, 1812433253ul>::operator()()
0.      0.     6.114 100.00   __libc_start_main
```

v0.1

Moving to Permuted Congruential Generator (pcg32)




```
main.cpp › int main( )
```

```cpp
std::vector<std::vector<CellState>> grid(grid_size,
                                         std::vector<CellState>(grid_size));

std::vector<std::vector<CellState>> next(grid_size,
                                         std::vector<CellState>(grid_size));

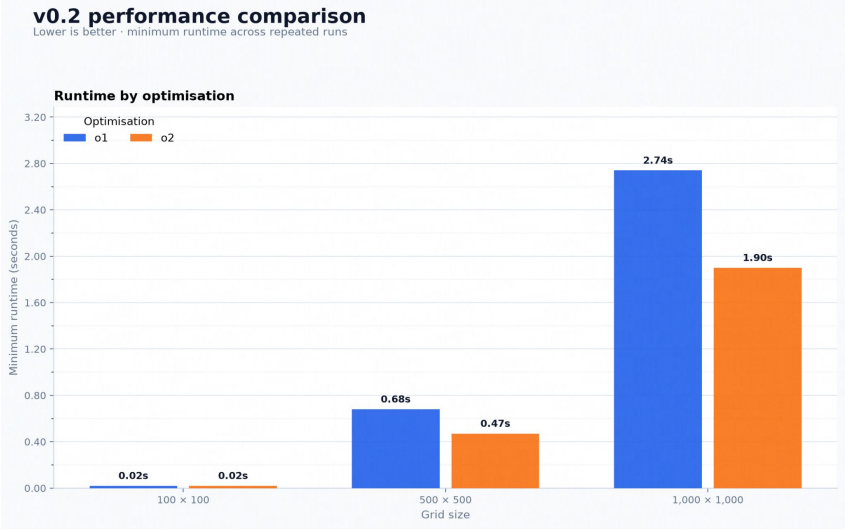
pcg32 rng(80085);
// TODO : change this to fix the cache access patterns
for (int t = 0; t < timesteps; ++t) {
  for (int y = 1; y < grid_size - 1; ++y) {
    for (int x = 1; x < grid_size - 1; ++x) {
      auto r = rng.nextFloat();
      switch (grid[x][y]) {
      case CellState::Empty:
        if (r < p) {
          next[x][y] = CellState::Tree;
        } else {
          next[x][y] = CellState::Empty;
        }
        break;
      case CellState::Burning:
        next[x][y] = CellState::Empty;
        break;
      case CellState::Tree:
        const auto up = grid[x][y - 1];
        const auto down = grid[x][y + 1];
        const auto left = grid[x - 1][y];
        const auto right = grid[x + 1][y];

        if (up == CellState::Burning || down == CellState::Burning ||
            left == CellState::Burning || right == CellState::Burning) {
          next[x][y] = CellState::Burning;
        } else if (r < f) {
          next[x][y] = CellState::Burning;
        } else {
          next[x][y] = CellState::Tree;
        }
        break;
      }
    }
  }
```

v0.2

Fixing cache access patterns

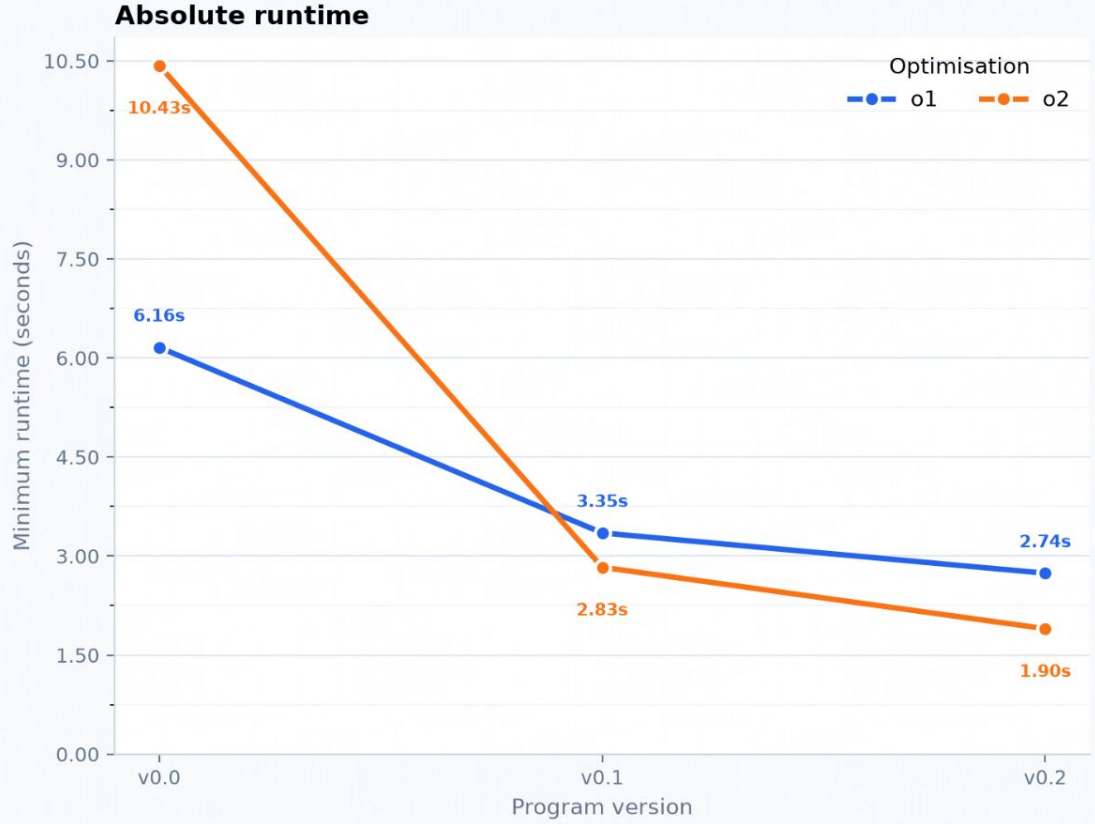
```
for (int y = 1; y < grid_size - 1; ++y) {
  for (int x = 1; x < grid_size - 1; ++x) {
      auto r = rng.nextFloat();
      switch (grid[x][y]) {
```

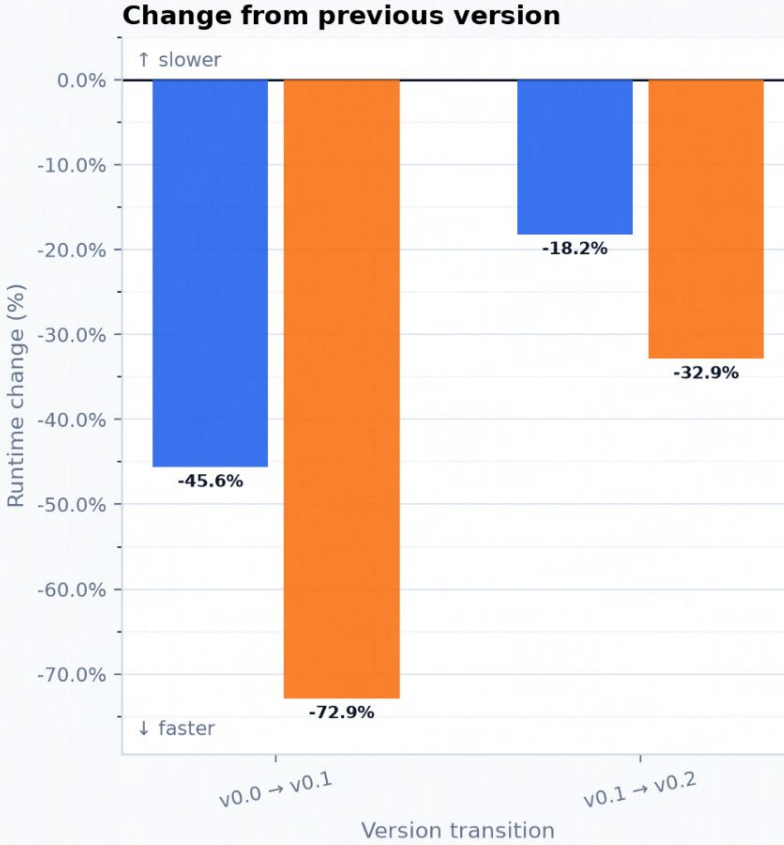



```
for (int x = 1; x < grid_size - 1; ++x) {
  for (int y = 1; y < grid_size - 1; ++y) {
      auto r = rng.nextFloat();
      switch (grid[x][y]) {
```

1000 × 1000 grid performance

Lower is better · minimum runtime across repeated runs





v0.2 - inspecting cache access patterns

```
53  Cellular automata based forest fire model
54  ------------------------------------------------
55  Model configuration
56    Grid size:       1002
57    Time steps:      500
58    p (growth):      0.02
59    f (lightning):   0.01
60  ------------------------------------------------
61  Build information
62    Version          0.1 better-random
63    Compiled:        Aug 23 2026 17:21:03
64    Compiler:        GCC 8.5.0 20210514 (Red Hat 8.5.0-28)
65    C++ standard:    C++17 (__cplusplus=201703)
66    Compiler target: x86_64-redhat-linux
67    Executable:      ./builds/v0.1/fire-v0.1-o2
68    Build flags:     O2 NDEBUG
69    Optimisation:    enabled
70
71  ==370295==
72  ==370295== I refs:        16,307,043,645
73  ==370295== I1  misses:             2,363
74  ==370295== LLi misses:             2,149
75  ==370295== I1  miss rate:           0.00%
76  ==370295== LLi miss rate:           0.00%
77  ==370295==
78  ==370295== D refs:         4,218,063,899  (3,217,584,929 rd   + 1,000,478,970 wr)
79  ==370295== D1  misses:     1,380,071,026  (  880,035,546 rd   +   500,035,480 wr)
80  ==370295== LLd misses:            43,547  (        9,027 rd   +        34,520 wr)
81  ==370295== D1  miss rate:           32.7% (         27.4%     +          50.0%  )
82  ==370295== LLd miss rate:            0.0% (          0.0%     +           0.0%  )
83  ==370295==
```

```
Cellular automata based forest fire model
------------------------------------------------
Model configuration
  Grid size:       1002
  Time steps:      500
  p (growth):      0.02
  f (lightning):   0.01
------------------------------------------------
Build information
  Version          0.2 better-random
  Compiled:        Aug 23 2026 18:57:28
  Compiler:        GCC 8.5.0 20210514 (Red Hat 8.5.0-28)
  C++ standard:    C++17 (__cplusplus=201703)
  Compiler target: x86_64-redhat-linux
  Executable:      ./bin/fire-v0.2-o2
  Build flags:     O2 NDEBUG
  Optimisation:    enabled

==370180==
==370180== I refs:        15,306,537,823
==370180== I1  misses:             2,369
==370180== LLi misses:             2,149
==370180== I1  miss rate:           0.00%
==370180== LLi miss rate:           0.00%
==370180==
==370180== D refs:         3,718,558,847  (2,718,079,876 rd   + 1,000,478,971 wr)
==370180== D1  misses:        16,452,188  (    8,416,707 rd   +     8,035,481 wr)
==370180== LLd misses:            43,549  (        9,025 rd   +        34,524 wr)
==370180== D1  miss rate:            0.4% (          0.3%     +           0.8%  )
==370180== LLd miss rate:            0.0% (          0.0%     +           0.0%  )
==370180==
==370180== LL refs:           16,454,557  (    8,419,076 rd   +     8,035,481 wr)
==370180== LL misses:             45,698  (       11,174 rd   +        34,524 wr)
==370180== LL miss rate:             0.0% (          0.0%     +           0.0%  )
```

v0.3 - easy wins

v0.2

main.cpp › int main( )

```cpp
std::vector<std::vector<CellState>> grid(grid_size,
                                         std::vector<CellState>(grid_size));

std::vector<std::vector<CellState>> next(grid_size,
                                         std::vector<CellState>(grid_size));
pcg32 rng(80085);
// TODO : change this to fix the cache access patterns
for (int t = 0; t < timesteps; ++t) {
  for (int y = 1; y < grid_size - 1; ++y) {
    for (int x = 1; x < grid_size - 1; ++x) {
      auto r = rng.nextFloat();
      switch (grid[x][y]) {
      case CellState::Empty:
        if (r < p) {
          next[x][y] = CellState::Tree;
        } else {
          next[x][y] = CellState::Empty;
        }
        break;
      case CellState::Burning:
        next[x][y] = CellState::Empty;
        break;
      case CellState::Tree:
        const auto up = grid[x][y - 1];
        const auto down = grid[x][y + 1];
        const auto left = grid[x - 1][y];
        const auto right = grid[x + 1][y];

        if (up == CellState::Burning || down == CellState::Burning ||
            left == CellState::Burning || right == CellState::Burning) {
          next[x][y] = CellState::Burning;
```

v0.3

main.cpp › int main( )

```cpp
for (int t = 0; t < timesteps; ++t) {
  for (int x = 1; x < grid_size - 1; ++x) {
    for (int y = 1; y < grid_size - 1; ++y) {
      switch (grid[x][y]) {
      case CellState::Empty:
        if (rng.nextFloat() < p) {
          next[x][y] = CellState::Tree;
        } else {
          next[x][y] = CellState::Empty;
        }
        break;
      case CellState::Burning:
        next[x][y] = CellState::Empty;
        break;
      case CellState::Tree:
        const auto up = grid[x][y - 1];
        const auto down = grid[x][y + 1];
        const auto left = grid[x - 1][y];
        const auto right = grid[x + 1][y];

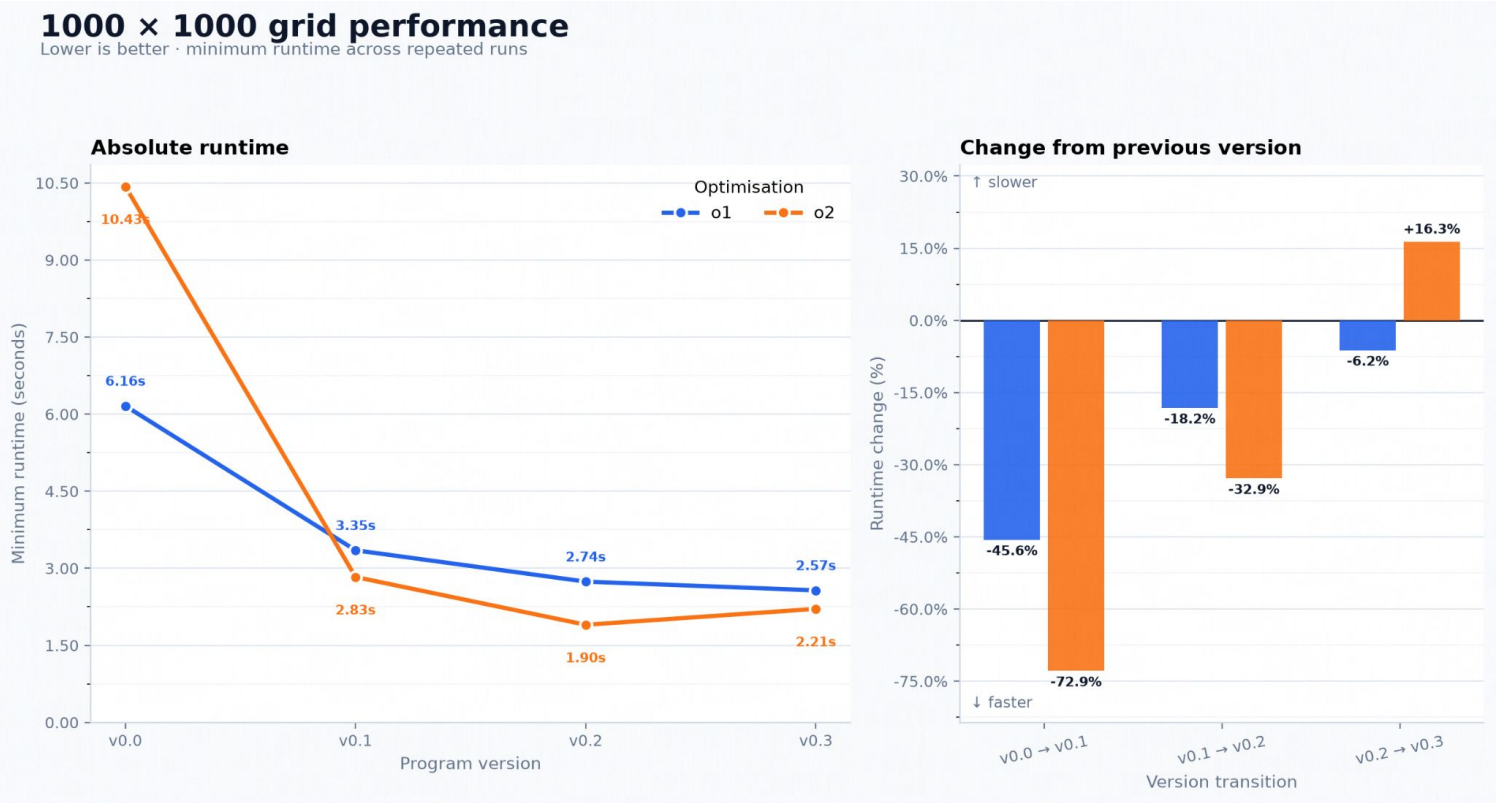
        if (up == CellState::Burning || down == CellState::Burning ||
            left == CellState::Burning || right == CellState::Burning)
          next[x][y] = CellState::Burning;
        } else {
          if (rng.nextFloat() < f) {
            next[x][y] = CellState::Burning;
          } else {
            next[x][y] = CellState::Tree;
          }
        }
        break;
      }
    }
  }
```

v0.3 - easy wins



[SC] [3] “The ‘Gru With a Gun’ Meme and the ‘Gru No’ Meme Come From a Doki Doki Literature Club Meme. : r/TheMatpatEffect.” Accessed: Aug. 27, 2026. [Online]. Available: https://www.reddit.com/r/TheMatpatEffect/comments/1rpku7c/the_gru_with_a_gun_meme_and_the_gru_no_meme_come/

v0.3 - the numbers



v0.4

- pcg32::next_float() actually calls pcg32::next_uint() internally. [3].

```
/// Generate a single precision floating point value on the interval [0, 1)
float nextFloat() {
  /* Trick from MTGP: generate an uniformly distributed
     single precision number in [1,2) and subtract 1. */
  union {
    uint32_t u;
    float f;
  } x;
  x.u = (nextUInt() >> 9) | 0x3f800000u;
  return x.f - 1.0f;
}
```

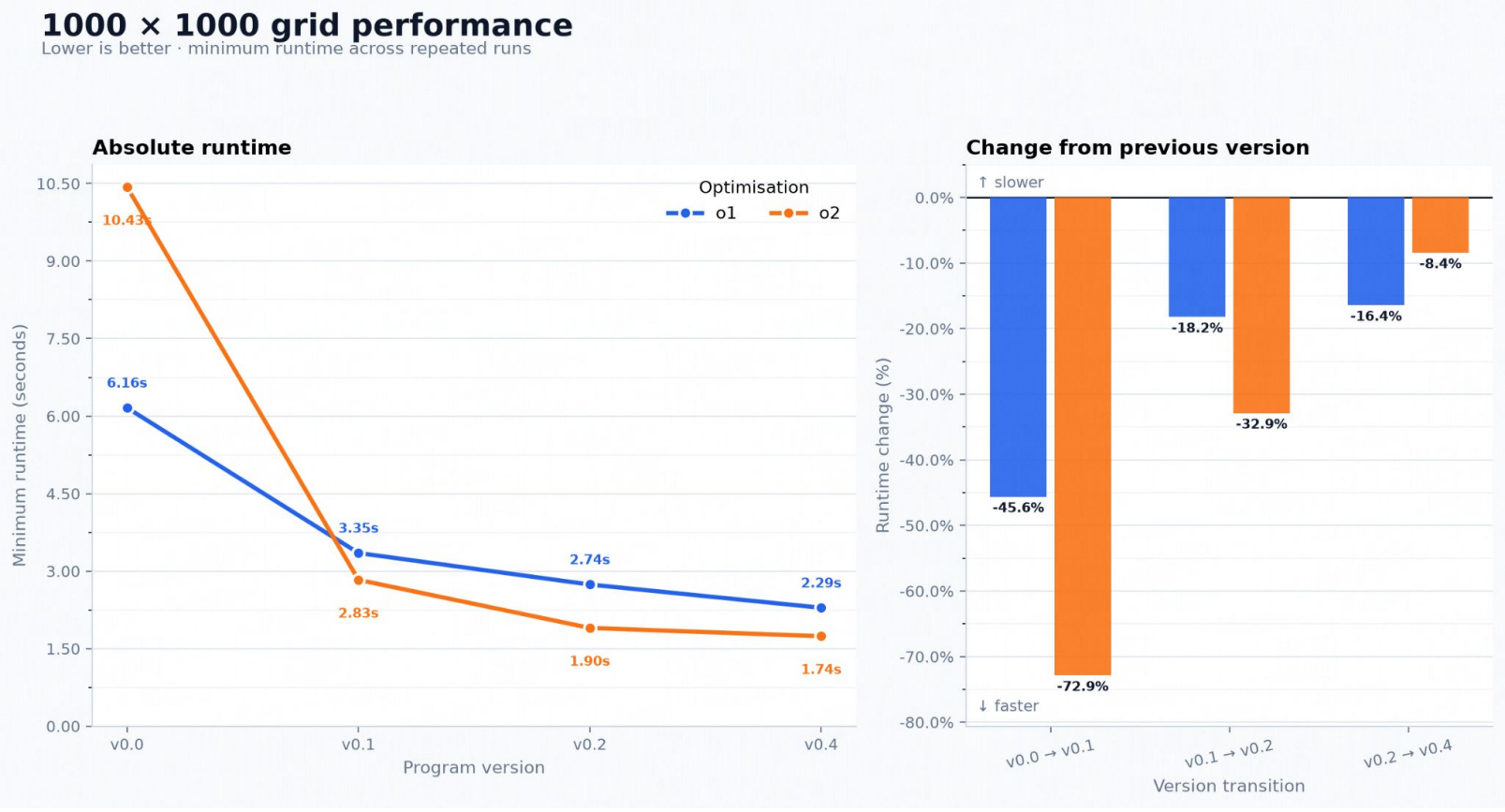
```
const uint32_t p_threshold = static_cast<uint32_t>(
    p * static_cast<double>(std::numeric_limits<uint32_t>::max()));

const uint32_t f_threshold = static_cast<uint32_t>(
    f * static_cast<double>(std::numeric_limits<uint32_t>::max()));

build_info::print_startup_banner(argv[0], grid_size, timesteps, p, f, VERSION,
                                 VERSION_NAME);
```

[4]W. Jakob, wjakob/pcg32. (Jul. 23, 2026). C++. Accessed: Aug. 27, 2026. [Online]. Available: https://github.com/wjakob/pcg32

v0.4 - the numbers



v0.5 - moving from 2d grid to a 1d grid

```
std::vector<std::vector<CellState>> grid(grid_size,
                                        std::vector<CellState>(grid_size));

std::vector<std::vector<CellState>> next(grid_size,
                                        std::vector<CellState>(grid_size));
```



```
std::vector<CellState> grid(grid_size * grid_size);

std::vector<CellState> next(grid_size * grid_size);
pcg32 rng(80085);
for (int t = 0; t < timesteps; ++t) {
  for (int x = 1; x < grid_size - 1; ++x) {
    for (int y = 1; y < grid_size - 1; ++y) {
      auto r = rng.nextUInt();
      const auto index = x * grid_size + y;
```

v0.5 - cache behaviour

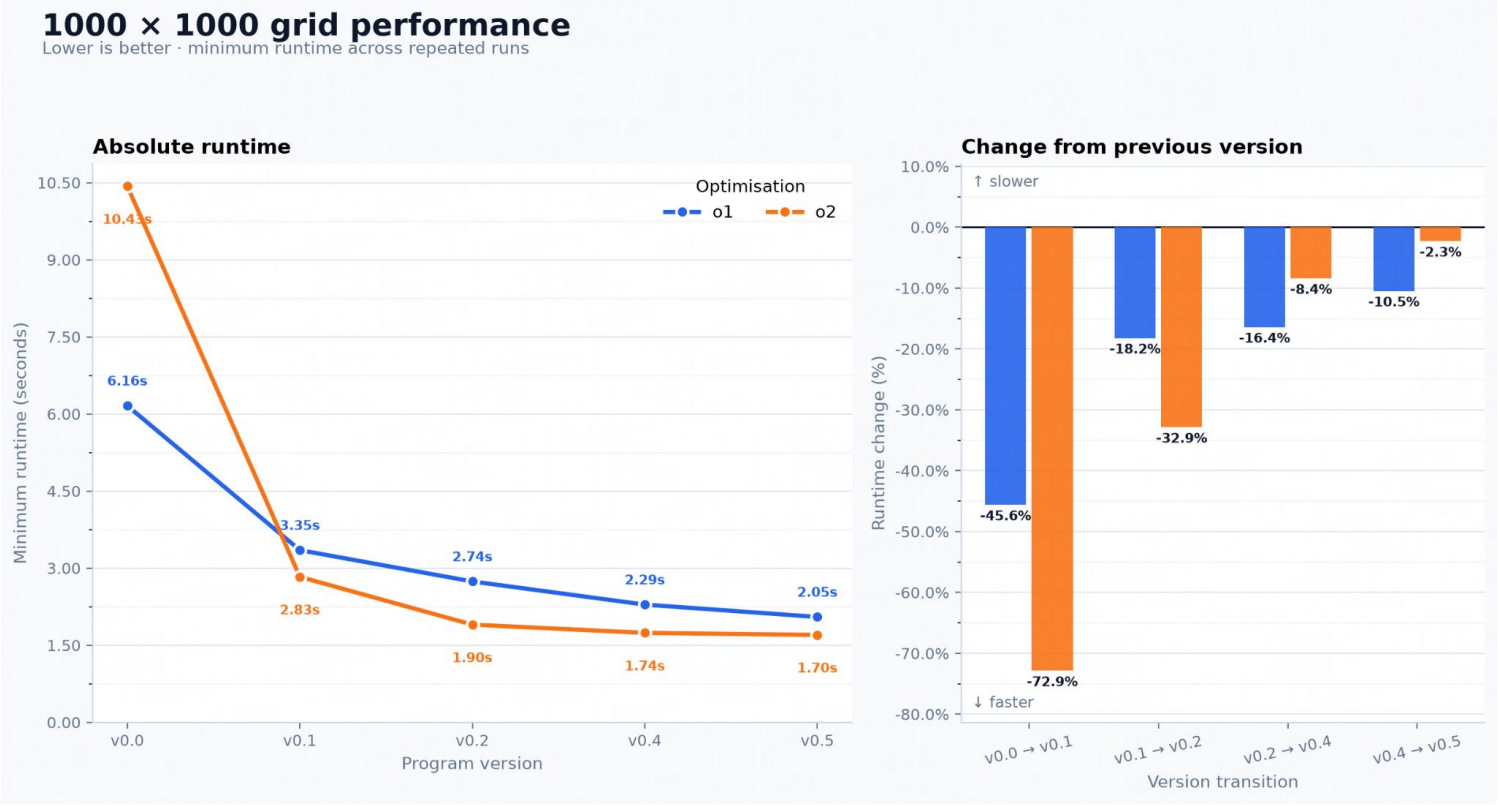
```
Cellular automata based forest fire model
-------------------------------------------------
Model configuration
  Grid size:       1002
  Time steps:      500
  p (growth):      0.02
  f (lightning):   0.01
-------------------------------------------------
Build information
  Version          0.4 rollbacks-and-int-compares
  Compiled:        Aug 24 2026 12:29:14
  Compiler:        GCC 8.5.0 20210514 (Red Hat 8.5.0-28)
  C++ standard:    C++17 (__cplusplus=201703)
  Compiler target: x86_64-redhat-linux
  Executable:      ./builds/v0.4/fire-v0.4-o2
  Build flags:     O2 NDEBUG
  Optimisation:    enabled

==824999==
==824999== I refs:        15,306,538,202
==824999== I1  misses:             2,370
==824999== LLi misses:             2,150
==824999== I1  miss rate:           0.00%
==824999== LLi miss rate:           0.00%
==824999==
==824999== D refs:         3,718,558,988  (2,718,079,910 rd   + 1,000,479,078 wr)
==824999== D1  misses:        16,450,901  (    8,414,940 rd   +     8,035,961 wr)
==824999== LLd misses:            43,559  (        9,033 rd   +        34,526 wr)
==824999== D1  miss rate:            0.4% (          0.3%     +           0.8%  )
==824999== LLd miss rate:            0.0% (          0.0%     +           0.0%  )
==824999==
==824999== LL refs:           16,453,271  (    8,417,310 rd   +     8,035,961 wr)
==824999== LL misses:             45,709  (       11,183 rd   +        34,526 wr)
==824999== LL miss rate:             0.0% (          0.0%     +           0.0%  )
```

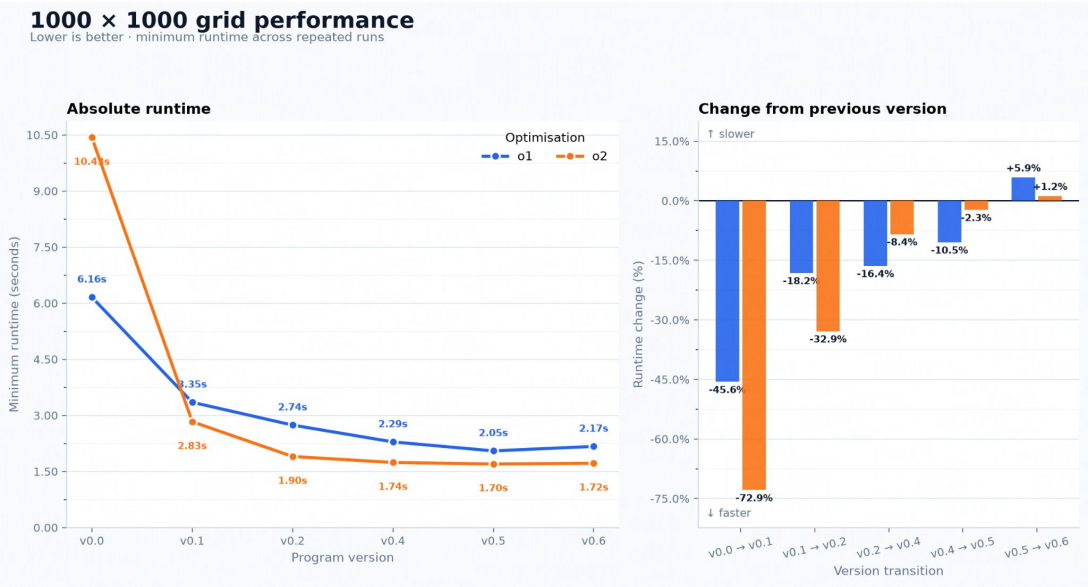
```
Model configuration
  Grid size:       1002
  Time steps:      500
  p (growth):      0.02
  f (lightning):   0.01
-------------------------------------------------
Build information
  Version          0.5 1d-grid
  Compiled:        Aug 24 2026 15:58:16
  Compiler:        GCC 8.5.0 20210514 (Red Hat 8.5.0-28)
  C++ standard:    C++17 (__cplusplus=201703)
  Compiler target: x86_64-redhat-linux
  Executable:      ./bin/fire-v0.5-o2
  Build flags:     O2 NDEBUG
  Optimisation:    enabled

==825093==
==825093== I refs:        12,080,391,259
==825093== I1  misses:             2,316
==825093== LLi misses:             2,121
==825093== I1  miss rate:           0.00%
==825093== LLi miss rate:           0.00%
==825093==
==825093== D refs:         1,731,465,834  (1,229,190,102 rd   + 502,275,732 wr)
==825093== D1  misses:        15,726,269  (    7,863,302 rd   +   7,862,967 wr)
==825093== LLd misses:            42,102  (        9,034 rd   +      33,068 wr)
==825093== D1  miss rate:            0.9% (          0.6%     +         1.6%  )
==825093== LLd miss rate:            0.0% (          0.0%     +         0.0%  )
==825093==
==825093== LL refs:           15,728,585  (    7,865,618 rd   +   7,862,967 wr)
==825093== LL misses:             44,223  (       11,155 rd   +      33,068 wr)
==825093== LL miss rate:             0.0% (          0.0%     +         0.0%  )
=== cachegrind end: ./bin/fire-v0.5-o2 ===
```

v0.5 - the numbers



v0.6-things that didn't work




```
pcg32 rng(80085);
for (int t = 0; t < timesteps; ++t) {
  for (int x = 1; x < grid_size - 1; ++x) {
    for (int y = 1; y < grid_size - 1; ++y) {
      auto r = rng.nextUInt();
      const auto index = x * grid_size + y;
```

```
pcg32 rng(80085);
for (int t = 0; t < timesteps; ++t) {
  for (int x = 1; x < grid_size - 1; ++x) {
    const int row_start = x * grid_size;
    const int row_end = row_start + grid_size - 1;
    for (int index = row_start + 1; index < row_end; ++index) {
      auto r = rng.nextUInt();
      switch (grid[index]) {
```

v0.7 - more things that didn't work

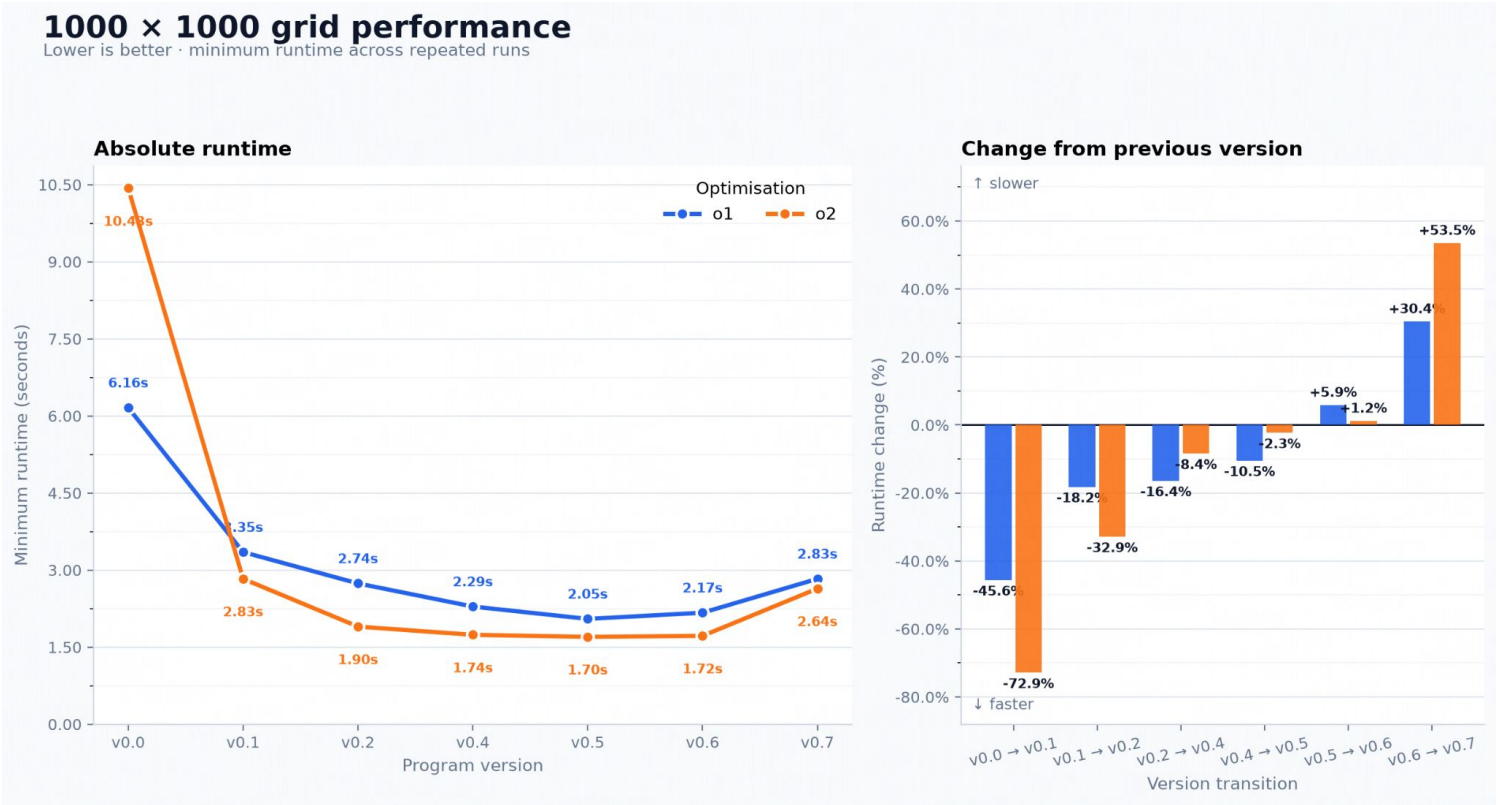
```
Model configuration
  Grid size:       1002
  Time steps:      500
  p (growth):      0.02
  f (lightning):   0.01
-------------------------------------------------
Build information
  Version          0.5 1d-grid
  Compiled:        Aug 24 2026 15:58:16
  Compiler:        GCC 8.5.0 20210514 (Red Hat 8.5.0-28)
  C++ standard:    C++17 (__cplusplus=201703)
  Compiler target: x86_64-redhat-linux
  Executable:      ./bin/fire-v0.5-o2
  Build flags:     O2 NDEBUG
  Optimisation:    enabled

==825093==
==825093== I refs:        12,080,391,259
==825093== I1  misses:             2,316
==825093== LLi misses:             2,121
==825093== I1  miss rate:           0.00%
==825093== LLi miss rate:           0.00%
==825093==
==825093== D refs:         1,731,465,834  (1,229,190,102 rd   + 502,275,732 wr)
==825093== D1  misses:        15,726,269  (    7,863,302 rd   +   7,862,967 wr)
==825093== LLd misses:            42,102  (        9,034 rd   +      33,068 wr)
==825093== D1  miss rate:            0.9% (          0.6%     +         1.6%  )
==825093== LLd miss rate:            0.0% (          0.0%     +         0.0%  )
==825093==
==825093== LL refs:           15,728,585  (    7,865,618 rd   +   7,862,967 wr)
==825093== LL misses:             44,223  (       11,155 rd   +      33,068 wr)
==825093== LL miss rate:             0.0% (          0.0%     +         0.0%  )
=== cachegrind end: ./bin/fire-v0.5-o2 ===
```

data/v0.7/cache-test/cache-test-565587.out

```
13  Model configuration
14    Grid size:       1002
15    Time steps:      500
16    p (growth):      0.02
17    f (lightning):   0.01
18   -------------------------------------------------
19  Build information
20    Version          0.7 packed_grid
21    Compiled:        Aug 24 2026 18:31:38
22    Compiler:        GCC 8.5.0 20210514 (Red Hat 8.5.0-28)
23    C++ standard:    C++17 (__cplusplus=201703)
24    Compiler target: x86_64-redhat-linux
25    Executable:      ./bin/fire-v0.7-o2
26    Build flags:     O2 NDEBUG
27    Optimisation:    enabled
28
29   ==834986==
30   ==834986== I refs:        24,361,357,966
31   ==834986== I1  misses:             2,328
32   ==834986== LLi misses:             2,129
33   ==834986== I1  miss rate:           0.00%
34   ==834986== LLi miss rate:           0.00%
35   ==834986==
36   ==834986== D refs:         3,837,502,582  (3,222,612,025 rd   + 614,890,557 wr)
37   ==834986== D1  misses:        3,948,761  (    3,938,327 rd   +      10,434 wr)
38   ==834986== LLd misses:        9,573  (        9,042 rd   +       9,531 wr)
39   ==834986== D1  miss rate:           0.1% (          0.1%     +         0.0%  )
40   ==834986== LLd miss rate:           0.0% (          0.0%     +         0.0%  )
41   ==834986==
42   ==834986== LL refs:           3,951,089  (    3,940,655 rd   +      10,434 wr)
43   ==834986== LL misses:            20,702  (       11,171 rd   +       9,531 wr)
44   ==834986== LL miss rate:            0.0% (          0.0%     +         0.0%  )
45   === cachegrind end: ./bin/fire-v0.7-o2 ===
46
```

v0.7-the numbers



Conclusion

Key result

- 81M → 294M cell updates/sec (single core, -O2). About ~3.5x improvement in absolute throughput

Most surprising

- Identical source code ran ~80% slower at -O2 than -O1 on GCC 8.5, but fine on GCC 15 — a reminder that source code and the machine code it becomes aren't the same thing
- Doing strictly less work (v0.3: fewer RNG calls) and using 4x less memory (v0.7: 2-bit packing) both made the program slower — fewer operations or less memory traffic doesn't automatically mean less time

AI Disclosure

AI tools like chatgpt5.6-luna and claude-sonnet-5 were used to help with producing this assignment. I used ai tools to

- create the the scripts that visualised the simulation and benchmarks
- understand and analyze the benchmark data
- write slurm scripts to submit jobs
- polish my script for the presentation

Bibliography

[1] “Forest-fire Model.” Accessed: Aug. 27, 2026. [Online]. Available: https://itp.uni-frankfurt.de/~gros/StudentProjects/Projects_2020/projekt_lars_dingeldein/

[2] B. Drossel and F. Schwabl, “Self-organized critical forest-fire model,” Phys. Rev. Lett., vol. 69, no. 11, pp. 1629–1632, Sep. 1992, doi: 10.1103/PhysRevLett.69.1629.

[3] “The ‘Gru With a Gun’ Meme and the ‘Gru No’ Meme Come From a Doki Doki Literature Club Meme. : r/TheMatpatEffect.” Accessed: Aug. 27, 2026. [Online]. Available: https://www.reddit.com/r/TheMatpatEffect/comments/1rpku7c/the_gru_with_a_gun_meme_and_the_gru_no_meme_come/

[4] W. Jakob, wjakob/pcg32. (Jul. 23, 2026). C++. Accessed: Aug. 27, 2026. [Online]. Available: https://github.com/wjakob/pcg32